Artificial Intelligence and Applications Lab Practical File

Sree Saicharan Vadapalli
Information Technology
ET23BTIT133

February 23, 2026

Contents

1 Analysis of Major Projects in AI **2**

1.1 Deep Blue . . . 2

1.1.1 Historical Background and Development . . . 2

1.1.2 Algorithmic Design and Evaluation Strategy . . . 3

1.1.3 System Architecture . . . 4

1.1.4 Performance, Limitations and Impact . . . 5

1.2 Chinook . . . 5

1.2.1 Historical Background and Development . . . 5

1.2.2 Computational Characteristics of English Draughts . . . 6

1.2.3 Algorithmic Design and Evaluation Function . . . 6

1.2.4 System Architecture . . . 7

1.2.5 Performance, Limitation and Impact . . . 7

1.3 ALVINN . . . 8

1.3.1 Historical Background and Development . . . 8

1.3.2 Model Architecture and Training Methodology . . . 8

1.3.3 Performance, Limitations and Impact . . . 9

1.4 ELIZA and ChatGPT . . . 9

1.4.1 Historical Background and Development . . . 9

1.4.2 Model Architecture and Training Methodology . . . 10

1.4.3 Performance, Limitations and Impact . . . 11

Practical 1

Analysis of Major Projects in AI

1.1 Deep Blue

In May 1997, IBM's Deep Blue, a chess-playing supercomputer designed by computer scientist Feng-hsiung Hsu, defeated the reigning world chess champion Garry Kasparov under standard tournament controls. Deep Blue disproved the assumption that humans were the strongest chess-playing entities. It was the first computer to win a game and a match against a reigning world champion under standard time controls. Deep Blue was a breakthrough in computer chess and high-performance search algorithms. IBM's first supercomputer, Stretch had a processing speed of 500,000 floating-point operations (FLOP). Deep Blue was capable of evaluating 200 *million* positions per-second, achieving a processing speed of 11.38 *billion* FLOPs per-second. Deep Blue heavily relied on **game-tree search** and hand-made evaluation heuristics instead of modern machine learning techniques. IBM's primary rationale behind creating a chess-playing supercomputer was to determine whether a computational search system could outperform human expertise. TESTING TESTING TESTING

1.1.1 Historical Background and Development

Feng-hsiung Hsu, a doctoral student at Carnegie Mellon University, began development of a chess-playing supercomputer under the name of ChipTest and even won the North American Computer Chess Championship in 1987. In 1988, Hsu and his team created a successor to ChipTest and named it "Deep Thought". Deep Thought, the reigning world computer chess champion of its time, lost a two-game exhibition match to the Russian grandmaster Garry Kasparov in 1989. After the loss, IBM renamed the chess machine as "Deep Blue", which was a play on IBM's nickname "Big Blue" at the time. Hsu and Murray Campbell hired Joel Benjamin to assist with the preparations for Deep Blue's matches against Kasparov. Deep Blue went on to play in the eight world computer chess championship and lost to Fritz (German Chess Program) in round five despite playing as white. Deep Blue played Kasparov twice more after Deep Thought's loss in 1989, losing both. Before it faced Kasparov again in May 1997, IBM upgraded Deep Blue's hardware, doubling its speed. During the six-game rematch, Deep Blue secured a victory in the deciding game with the score $3\frac{1}{2}$–$2\frac{1}{2}$ against Kasparov. Thus, it became the first computer system to defeat the reigning world champion in a match under standard time controls. The version of Deep Blue that was victorious against Kasparov searched to a depth of six to eight moves and around 20 moves in some situations. Kasparov demanded a rematch, but IBM had dismantled Deep Blue after its victory and refused the rematch, which remains a subject of controversy in chess community. Since Deep Blue's victory, computer scientists have developed software for complex board games, the AlphaGo series (AlphaGo, AlphaGo Zero, AlphaZero) defeated top Go players in 2016–2017.

1.1.2 Algorithmic Design and Evaluation Strategy

Unlike modern learning-based systems, Deep Blue relied on deterministic search and expert-designed heuristics instead of statistical training. It derived its strength primarly from a large and complex game-tree search combined with handcrafted evaluation function (set of rules and mathematical weights). Before the modern AI era, computers didn't "learn" good positions in the traditional sense. They're mostly considered to be a form of Symbolic AI or Good Old Fashioned AI (GOFAI).

Tree Search

In computer science, a search tree is a dynamic, hierarchical data structure. It's represented as a directed graph of nodes (also called vertices or states) connected by directed edges (also called arcs or state transitions). A chess game can be represented as a tree search problem because a chess game is deterministic, has perfect information and is zero-sum. The states are the particular arrangements of pieces on the chess board and the available actions are the possible legal chess moves for the current player in a specfic arrangement. In the game of chess, nodes represent the altering white and black "to move" chess positions and directed edges represent the alternative white and black moves.

Evaluation Function

Shannon's paper describes evaluation function as the primary ingredient in a chess-playing program. It is a type of function that takes in the current state of the game and reduces it to an estimated scalar (numerical) value of the state that is used for evaluating that state.

$$\text{Score} = \sum_i w_i \, f_i(\text{position})$$

where:

$$f_i \text{ represents positional features,}$$

$$w_i \text{ represents corresponding weights.}$$

Minimax and Alpha-Beta Pruning

Deep Blue implmented a minimax search framework apt for deterministic, zero-sum and perfect-information games such as chess. The minimax program made the following assumptions:

- The maximizing player selects moves to maximize the evaluation score.
- The minimizing player (opponent) selects moves to minimize the score.

In simple terms, Minimax looks ahead as many steps as the computing power allows for each move and examine all the possible moves the opponent could make in each of the opponent's future turns, given that the original move is made. It computes the maximum loss that the opponent could induce via their moves and attempts to choose the move that minimizes it the most.The formal recursive definition of the minimax algorithm is:

$$V(s) = \begin{cases} \text{Utility}(s) & \text{if } s \text{ is terminal} \\ \max\limits_{a \in \text{Actions}(s)} V(\text{Result}(s,a)) & \text{if } s \text{ is Max node} \\ \min\limits_{a \in \text{Actions}(s)} V(\text{Result}(s,a)) & \text{if } s \text{ is Min node} \end{cases}$$

where:

$s \in S$	game state
$S_T \subseteq S$	set of terminal states
$S_{\text{Max}} \subseteq S$	states where Max plays
$S_{\text{Min}} \subseteq S$	states where Min plays
$A(s)$	legal actions in state s
$T(s, a)$	transition function
$U(s)$	utility function

Additionally, Deep Blue utilized Alpha-Beta pruning to elimate branches that had minimal to no effect on the final decision because the size of a full chess game tree ($Branching\ Factor \approx 35$ and $Shannon\ Number \approx 10^{120}$) makes it computationally unmanageable. Alpha-beta pruning maintains two bounds:

- **Alpha** (α)**:** Best value found so-far for the maximizing player.
- **Beta** (β)**:** Best value found so-far for the minimizing player.

Exploration of a branch is pruned or considered unnecessary when $\alpha \geq \beta$. This substantially reduces the **effective branching factor** and enables deeper search. It is important to note that alpha-beta pruning is implemented using Depth-First Search (DFS), meaning it only one path from the root to the current position needs to be kept in memory. It introduces α and β values to the minimax value.

Transposition Tables

Deep Blue used transposition tables to store previously evaluated positions because many chess positions can be reached through a different order of moves. Deep Blue used Zobrist hashing to hash the positions, allowing constant-lookup time and avoiding redundant computation.

1.1.3 System Architecture

Deep Blue's computational dominance was enabled by specialized hardware and large-scale parallelization. The system represented a co-design of software search algorithms and domain-specific hardware acceleration. The 1997 version of Deep Blue consisted of:

1. 30 IBM RS/6000 SP nodes
2. 480 custom VLSI chess chips
3. High-speed interconnect for distributed computation

Each node had custom chess processors that were capable of: generating legal moves in hardware, evaluating positions and performing search acceleration operations. Unlike general-purpose CPUs, these chips were specifically designed for chess computation. This domain specific design allowed Deep Blue to outperform pure software-based systems.

Parallel Search Strategy

It implemented parallel search strategy using a master-slave architecture. Different processors could explore different legal moves from root position simultaneously, which reduced the overall time taken to reach a decision. This is also called **root level parallelism**. Search trees were dynamically partitioned among processors and synchronization was essential to maintain correct alpha-beta bounds across the nodes. Efficient synchronization prevents; redundant exploration, bound inconsistencies, communication overhead.

1.1.4 Performance, Limitations and Impact

Performance

Deep Blue's performance was primarily driven by large-scale parallel search and hardware acceleration. The 1997 system evaluated approximately 200 million positions per second and achieved a peak performance of around 11.38 billion floating-point operations per second. In practical play, it typically searched to depths of six to eight moves in complex positions and up to twenty moves in tactical lines. Its strength did not come from floating-point computation alone, but from efficient alpha–beta pruning, strong move ordering, selective search extensions, and the use of custom VLSI chess chips across multiple processing nodes.

Limitations

Despite its success, Deep Blue was a highly specialized system designed exclusively for chess. It did not possess learning capabilities and could not adapt or improve through self-play. Its evaluation function was manually engineered and relied on predefined heuristics rather than statistical training. Furthermore, its strength depended heavily on massive computational resources and domain-specific hardware. Without this specialized infrastructure, its performance would have been significantly reduced.

Impact

Deep Blue's victory over Garry Kasparov marked a major milestone in artificial intelligence and computer chess. It demonstrated that large-scale computational search could surpass human expertise in a complex strategic domain. Later systems, such as AlphaZero, introduced learning-based approaches that fundamentally changed how game-playing AI systems were designed.

1.2 Chinook

In 1996, Microsoft introduced Chinook, a computer checkers program developed at the University of Alberta under the leadership of computer scientist Jonathan Schaeffer. Chinook challenged and later defeated the reigning world checkers champion Marion Tinsley, marking the first time a computer program competed for and won a human world championship title in any game. Unlike earlier systems that relied primarily on heuristic evaluation, Chinook combined high-performance game-tree search with large, expertly constructed endgame databases containing precomputed perfect-play positions. By 2007, after years of large-scale retrograde analysis, the team solved the game of checkers (English draughts), proving that perfect play from both sides results in a draw. Chinook demonstrated that exhaustive search, domain-specific heuristics, and massive endgame computation could not only rival but mathematically surpass human expertise in deterministic, perfect-information games.

1.2.1 Historical Background and Development

In 1989, Jonathan Schaeffer and his research team at the University of Alberta began developing Chinook, a computer program designed to compete at the highest levels of professional checkers. The project quickly advanced, and in 1992 Chinook earned the right to challenge reigning world champion Marion Tinsley. Although it narrowly lost that initial match, the event marked the first time a computer program competed for a human world championship title in any game.

A rematch was held in 1994, but Tinsley withdrew due to health reasons. Chinook subsequently competed against Don Lafferty for the vacant title and won, becoming the

first computer program to win a human world championship in a game of intellectual skill. Following its competitive success, the Chinook team continued refining the program with the long-term objective of fully solving checkers.

After years of computational analysis and database construction, the team announced in 2007 that the game of checkers had been weakly solved, proving that perfect play from both sides results in a draw. Chinook's competitive career was formally retired after this milestone, concluding nearly two decades of sustained research and development.

1.2.2 Computational Characteristics of English Draughts

The version of checkers addressed by Chinook is formally known as English draughts. From a computational perspective, it is a deterministic, perfect-information, two-player zero-sum game. Each of these properties has direct algorithmic implications.

These characteristics reduce the problem to adversarial search over a fully observable state space. There is no hidden information, no probability modeling, and no stochastic branching. The entire game can, in principle, be represented as a finite directed graph where; nodes represent legal positions on the board, edges represent legal moves and terminal nodes denote a win or loss or draw. This makes checkers mathematically well-defined and suitable for exhaustive analysis.

State Space and Game Tree Size

- **Estimated total legal positions** $\approx$ order of 10^{20}
- **Average branching factor** $\approx$ 8–10 moves per-position
- **Maximum game length:** bounded due to repetition and capture rules.

Although 10^{20} positions is enormous, it is dramatically smaller than chess ($\approx 10^{42}$ estimated positions) and vastly smaller than Go ($\approx 10^{170}$). This relative tractability is what made checkers a viable candidate for complete solution.

Why Checkers Was Solvable

The key insight is not that checkers is "simple," but that it is finite and computationally bounded. A game is solvable if it meets the following conditions:

1. Its state space can be fully enumerated or decomposed
2. Terminal outcomes can be propagated backward
3. Storage and compute resources are sufficient

Checkers met these conditions decades before chess or Go could realistically do so. The combination of moderate branching factor, forced-move constraints, and reducible endgame complexity made it the first non-trivial classical board game to be weakly solved at world-championship scale.

1.2.3 Algorithmic Design and Evaluation Function

The algorithmic foundation of Chinook was a depth-limited minimax search enhanced with alpha–beta pruning and iterative deepening to efficiently explore the adversarial game tree under tournament time constraints. To mitigate redundant computation, the system employed large transposition tables indexed via Zobrist hashing, enabling constant-time retrieval of previously evaluated positions. Move ordering heuristics were used to maximize

pruning effectiveness, particularly in tactically forced capture sequences. The evaluation function combined material balance, piece advancement, mobility, center control, tempo considerations, and structural formations, with parameters refined through extensive expert consultation and empirical tuning rather than machine learning. In late-game scenarios, heuristic evaluation was replaced by exact lookup into precomputed endgame databases generated through retrograde analysis, ensuring mathematically optimal play once positions entered solved regions. This hybrid design—forward search guided by domain-specific heuristics and terminated by exact database resolution—allowed Chinook to approximate optimal play in the middlegame while guaranteeing perfect decisions in endgame configurations.

1.2.4 System Architecture

The system architecture of Chinook was designed around high-performance sequential search augmented by large-scale persistent storage for endgame databases. The core engine executed on conventional high-end workstations, relying on optimized C implementations, efficient bitboard representations, and tightly controlled memory management to maximize search throughput per clock cycle. A layered architecture separated move generation, search control, evaluation, and database access, allowing clean integration of exact endgame lookups into the forward search pipeline. Transposition tables were maintained in RAM for rapid access, while larger solved-position databases were stored on disk with compressed indexing schemes to balance storage constraints and lookup latency. The architecture emphasized space–time tradeoffs: heavy precomputation and storage reduced runtime computation during tournament play. Unlike massively parallel supercomputing systems, Chinook's design prioritized algorithmic efficiency, caching, and structured decomposition of the game space over raw hardware scaling, reflecting the engineering goal of solving a finite adversarial domain within practical computational limits.

1.2.5 Performance, Limitation and Impact

Performance

Chinook achieved superhuman performance through deep alpha–beta search combined with exact endgame database lookups. In tournament play, it consistently evaluated millions of positions per move while leveraging cached transpositions to avoid redundant computation. Its strength was most pronounced in late-game scenarios, where database access guaranteed mathematically optimal decisions. This hybrid of heuristic-guided search and perfect endgame resolution enabled Chinook to compete at, and ultimately surpass, world-championship human performance.

Limitations

Despite its dominance, Chinook was fundamentally domain-specific and non-generalizable. Its architecture depended on the finite and relatively constrained state space of English draughts; the approach does not scale to vastly larger games such as chess or Go without prohibitive storage and computation costs. The system required extensive precomputation and long development cycles, making it impractical for dynamic or uncertain environments. It was an engineered solver, not an adaptive learning system.

Impact

Chinook's 2007 weak solution of checkers established the first complete resolution of a non-trivial classical board game at championship scale. The project demonstrated that exhaustive computation, when paired with rigorous state-space decomposition, can fully solve structured

adversarial domains. Beyond competitive play, it provided a proof-of-concept for large-scale retrograde analysis and influenced subsequent research in computational game theory, search optimization, and formal game solving.

1.3 ALVINN

In 1989, researchers at Carnegie Mellon University developed ALVINN (Autonomous Land Vehicle In a Neural Network), one of the earliest end-to-end neural network systems for autonomous driving. Created by Dean Pomerleau, ALVINN was designed to learn steering control directly from camera images using supervised training, rather than relying on manually programmed rules or symbolic reasoning. The system was deployed on the Navlab research vehicle and demonstrated the ability to steer a car autonomously on real roads at highway speeds. Unlike search-based game-playing systems of its era, ALVINN represented a shift toward connectionist approaches, mapping raw sensory input to motor output through learned internal representations. It marked an early and influential step toward modern deep learning–based autonomous driving systems.

1.3.1 Historical Background and Development

The development of ALVINN (Autonomous Land Vehicle In a Neural Network) emerged during the late 1980s within the autonomous vehicle research program at Carnegie Mellon University. At the time, most autonomous navigation systems relied on symbolic AI, rule-based perception pipelines, and handcrafted feature extraction. These systems required explicit modeling of lane markings, road geometry, and vehicle kinematics, resulting in brittle performance under real-world variability.

Under the direction of Dean Pomerleau, ALVINN pursued a fundamentally different paradigm: learning a direct mapping from camera input to steering commands using a neural network. Development began around 1988–1989 as part of the Navlab autonomous vehicle project. The system was trained using supervised learning, where human driving examples were recorded and used to adjust network weights via backpropagation. Rather than encoding explicit rules for lane detection, ALVINN learned internal representations sufficient to produce appropriate steering angles. The system was deployed on CMU's Navlab vehicle platform and demonstrated sustained autonomous driving on public highways, including extended stretches where the neural network controlled steering at typical traffic speeds. Subsequent iterations incorporated additional sensors and refinements to improve robustness under varying lighting and road conditions. ALVINN's development marked an early and influential shift from symbolic robotics toward data-driven control systems, anticipating the end-to-end learning architectures that would dominate autonomous driving research decades later.

1.3.2 Model Architecture and Training Methodology

The defining technical feature of ALVINN was its direct mapping from visual input to steering output without intermediate symbolic processing. Formally, the system learned a function:

$$f : x \to \theta$$

Unlike traditional piplines that decomposed perception into lane detection, feature extraction, and rule-based control, ALVINN collapsed the entire stack into a single learned nonlinear function approximator. The input consisted of a **downsized greyscale images** (32×32). The input was flattened into a vector and fed into a fully connected Multi-Layer Perceptron (MLP) with a single hidden layer and the network computation followed the standard feedforward structure. ALVINN used multiple output units corresponding to discrete

steering directions instead of single scalar steering angle. It was trained using backpropagation on human driving examples. This architecture was was computationally minimal yet conceptually radical for its time.

1.3.3 Performance, Limitations and Impact

Performance

ALVINN became renowned for demonstrating real-time autonomous steering on public roads in the 1980s. The system was successful at controlled steering at highway speeds under structured road conditions. It consisted of a single feedforward network, allowing low-latency control suitable for **closed-loop driving**.

Limitations

It's architecture was shallow, fully-connected and trained on limited data which constrained its representational capacity. It didn't explicitly exploit any spatial locality in images due the lack of a convolutional structure. Lighting changes, complex traffic environments, poorly marked roads led to significant performance degredation.

Impact

It's considered to be the earliest successful demonstration of **end-to-end learning** for real-world applications. It demonstrated that gradient-based neural networks could learn representations from the raw pixels of the image directly. Contemporary convolutional and transformer-based driving systems trace their lineage to ALVINN.

1.4 ELIZA and ChatGPT

In 1996, Joseph Weizenbaum developed ELIZA at MIT, which is considered to be one of the earliest conversational computer programs. It generated responses through **pattern matching** and **predefined rules**. Decades later, in 2022, an American artificial intelligence research organization called OpenAI introduced a generative artificial intelligence chatbot "ChatGPT" that could generate text, speech and images in response to user prompts. ChatGPT generates language through **statistical learning** over massive text corpora, which is a fundamentally different ELIZA that relied upon predefined rules. ELIZA (Symbolic pattern-based system) and ChatGPT (Data-driven deep learning model) serve as representatives of two different eras in natural language processing that illustrate the evolution from handcrafted conversational scripts to large-scale probabilistic language modeling.

1.4.1 Historical Background and Development

Following its creation in 1966 by Joseph Weizenbaum at MIT, ELIZA became a focal point in early debates about machine understanding and human–computer interaction. Although technically simple, its reception exposed how readily users projected intent and comprehension onto computational systems. Weizenbaum himself grew critical of the enthusiasm surrounding it, arguing that superficial linguistic simulation should not be mistaken for genuine cognition. ELIZA's development occurred within a research culture centered on symbolic processing and rule-driven manipulation of text, reflecting the dominant AI methodology of the 1960s.

In contrast, ChatGPT, developed by OpenAI, emerged from decades of progress in statistical NLP, large-scale data collection, and high-performance computing. Its development

built upon advances in distributed training, large neural architectures, and alignment techniques that refined raw language modeling into interactive dialogue systems. The release of ChatGPT marked a transition from research prototypes to mass public deployment, bringing conversational AI into mainstream use and reshaping expectations about machine-generated language at global scale.

1.4.2 Model Architecture and Training Methodology

ELIZA's Rule Based Pattern Matching

It used deterministic pattern matching rather than statistical learning. It's architecture mainly consisted of the following:

1. Input parsing
2. Keyword detection
3. Pattern matching
4. Template substitution

Formally,

$$if \quad x \in P_i \implies y = T_i(x)$$

where:

$$\begin{aligned} x &= \text{user input string} \\ P_i &= \text{predefined pattern} \\ T_i &= \text{corresponding response template} \end{aligned}$$

ELIZA didn't have a learning phase, the mapping function was fixed to $f(x) = T_i(x)$ if a matching pattern P_i exists. Otherwise, a default fallback response was used.

ChatGPT's Transformer-Based Architecture

ChatGPT is based on a transformer architecture that models language as a conditional probability distribution over token sequences. I.e, it treats language generation as a **next-token prediction problem**. The model learns to estimate the probability of the next word or token given all previous words in a sequence. It's based **generative pretrained transformer** (GPT) models that have been fine-tuned for conversational assistance. The fine-tuning process involved supervised learning and reinforcement learning from human feedback (RHLF) using human trainers to improve the model performance. In the case of supervised learning, the trainers acted as both the users and the AI assistant. In the reinforcement learning stage, human trainers first ranked responses generated by the model in previous conversations and used these rakings to create **reward models** to fine-tune the model further using proximal policy optimization (PPO). Proximal Policy Optimization is a reinforcement learning algorithm training an intelligent agent. It is a policy gradient method that takes small policy update steps, so the agent can reach higher and higher rewards in expectation. It is important to note that a step size that is too big may direct the policy in a suboptimal direction, thus having a little possibility of recovery, while a step size that is too small lowers the overall efficiency. To combat this, PPO implements a **clip function** to place constraints on the policy update of an agent from being too large in-order to use larger step sizes without degrading the model performance. During pretraining, the model optimizes the objective:

$$\max_{\theta} \sum_{t} \log P_{\theta}(x_t \mid x_{<t})$$

This means it adjusts its parameters θ to maximize the likelihood of predicting each next token x_i, conditioned on the preceding tokens $x_{<t}$. Self-attention allows each word in a sentence to compute weighted relationships with every other word. This enables the model to capture long-range dependencies.

1.4.3 Performance, Limitations and Impact